

Leveraging Agentic AI Middleware for Intelligent Interaction and Modernization of Legacy Enterprise Applications

M.Z.M. Ayyash

Dept. of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22355850@my.sliit.lk

D.N. Rajapaksha

Dept. of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22311436@my.sliit.lk

M.A.M. Mufthi

Dept. of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22355478@my.sliit.lk

S.M.A.C.M. Rashad

Dept. of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22331472@my.sliit.lk

Mr. Vishan Jayasinghearachchi

Dept. of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
vishan.j@sliit.lk

Dr. Dharshana Kasthurirathna

Dept. of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
dharshana.k@sliit.lk

Abstract—Enterprise organizations across healthcare, finance, and government continue to operate mission-critical software on decades-old desktop platforms, spending over \$300 billion annually on maintenance while remaining unable to modernize due to source-code inaccessibility and prohibitive replacement costs [1]. This paper presents LAMA (Legacy Application Modernization Architecture), an agentic AI middleware framework that modernizes legacy Windows desktop applications through non-intrusive orchestration, requiring zero modification to the underlying system. LAMA integrates four principal components: an LLM-driven frontend generation pipeline that automatically constructs modern web interfaces from legacy screenshots; SOACRS, a self-optimizing coordination microservice that produces adaptive, fallback-aware execution plans using a hybrid rule-based and contextual bandit scoring engine; a stateful multi-agent orchestrator that enforces pre- and post-condition verification across dynamically composed automation workflows; and a computer-vision-augmented RPA bot layer combining PyAutoGUI, PyDirectInput, and AutoHotkey for reliable interaction with Win32-era graphical interfaces. The framework was evaluated against a Visual Basic 6 hospital management system running on a Windows XP virtual machine across 150 executions spanning three clinical workflows. Results demonstrate an overall task success rate of 97.0%, an OCR field-level accuracy of 94.1%, and a workflow recovery rate of 98.8%, with a mean end-to-end execution time of 7.8 seconds. All but three failures recovered autonomously through retry and fallback mechanisms, with no execution requiring full compensatory rollback.

Index Terms—legacy system modernization, robotic process automation, agentic AI, large language model, multi-agent systems, GUI automation, orchestration, computer vision, enterprise middleware

I. INTRODUCTION

Legacy enterprise software is not going away. Across healthcare, government, manufacturing, and financial services, organizations continue to depend on applications written in Visual Basic 6, Delphi, COBOL, and PowerBuilder — systems whose interfaces have not changed since the late 1990s but whose underlying logic is trusted, proven, and deeply embedded in daily operations. A 2023 Gartner analysis estimated that organizations collectively spend over \$300 billion per year maintaining these systems, with most of that expenditure going toward preventing failures rather than adding capabilities [1].

Replacing these systems is rarely practical. Developers seldom hand over source code at the end of an engagement, which means reconstruction requires reverse-engineering business logic from observed behavior — a process that is expensive and error-prone. The risk of disrupting operations during a migration keeps most organizations locked in maintenance mode indefinitely.

The physical architecture of legacy desktop applications compounds the problem. These applications are typically bound to specific operating systems and hardware, meaning staff in hospitals or government offices must physically access a dedicated machine for even routine tasks — registering a patient, scheduling an appointment, or processing a payment. That constraint creates bottlenecks and limits the flexibility that modern workflows demand.

Robotic Process Automation (RPA) has been used for years to extend legacy systems without touching their code. RPA bots drive the graphical interface the same way a human operator would, requiring no source-code access. The well-documented weakness of conventional RPA, however, is fragility: automations are built around hard-coded screen coordinates and fixed element selectors, so any change to the interface layout breaks the workflow. There is also no mechanism for handling variable or natural-language input — each bot executes a predetermined script and nothing more.

This paper describes LAMA, a framework built to address that fragility. Rather than replacing RPA, LAMA adds an AI-driven planning and orchestration layer above it. Users interact with a modern web interface generated automatically from screenshots of the legacy application. Their inputs are analyzed by a planning module that determines which automation bots are needed and in what order, and an orchestration engine coordinates those bots to execute the task against the live legacy system — while it continues running unmodified in a virtual machine.

The specific contributions of this work are as follows:

- A hybrid execution planning architecture combining LLM-based reasoning with a deterministic rule engine to produce reliable, fallback-aware GUI action plans.
- SOACRS, a lightweight Node.js microservice implementing a contextual-bandit-scored tool selection engine with explicit state-machine transitions for success, failure, and timeout.
- A stateful multi-agent orchestration engine that enforces pre- and post-condition verification at each step, with plug-and-play bot registration that requires no middleware restart.
- An empirical evaluation on a VB6 Hospital Management System under Windows XP, demonstrating 97.0% task success and 98.8% recovery across 150 trials with zero source-code modification.

II. RELATED WORK

A. Legacy System Modernization

Bennett’s 1995 analysis identified what he termed the paradox of success: the more reliably a system performs its core function, the more deeply it embeds into surrounding processes and the harder it becomes to replace [2]. Kazman, Kruchten, and Bennett later developed architecture reconstruction techniques that map hidden dependencies without full re-engineering [5], while Pérez-Castillo et al. demonstrated screen-scraping as a practical non-intrusive extension mechanism for mainframe systems in regulated environments [6]. Market data broadly supports the commercial case for non-intrusive alternatives: conventional modernization projects average 16 months, cost approximately \$1.5M, and fail or are abandoned 80% of the time [1].

B. Robotic Process Automation

Lacity and Willcocks confirmed RPA’s value for rule-based GUI tasks while specifically identifying selector fragility —

the tendency of hard-coded element locators to break under interface updates — as the dominant long-term operational challenge [3]. Ribeiro et al., reviewing 52 studies, found that AI-enhanced RPA substantially improves resilience to UI drift but that most implementations still lack higher-level coordination layers capable of managing multiple bots across multi-step workflows [7]. El-Gharib et al. demonstrated that process mining can identify optimal automation targets before deployment, improving RPA rollout efficiency [8]. Oberhauser and Pogolski explored partially self-healing RPA that recovers from minor layout changes without human intervention, but their approach did not extend to multi-agent coordination or API-level integration [4].

C. AI-Driven GUI Understanding

Beltramelli’s pix2code established the feasibility of automated GUI analysis without framework access, achieving 77% accuracy generating UI code from screenshots using a CNN-LSTM architecture [9]. The RICO dataset subsequently provided large-scale mobile UI training data [10]. More recently, OmniParser advanced the state of the art by parsing screenshots into structured element trees suitable for reliable action grounding [11]. UI-TARS demonstrated autonomous GUI agents operating entirely through native interfaces via visual understanding [12]. UI-TARS is the closest prior work to the RPA connector described in this paper; the key difference is that UI-TARS operates as a standalone agent, while our RPA connector is one component in a coordinated multi-agent pipeline with API-exposure capabilities.

D. Multi-Agent Coordination

Smith’s Contract Net Protocol formalized capability-based decentralized task allocation, in which agents advertise their capabilities and accept tasks through bidding [13]. Nii’s Blackboard Architecture introduced shared-memory coordination, where agents contribute to and read from a common data structure without direct communication [14]. BDI (Belief-Desire-Intention) models, developed by Bratman [15] and formalized for software agents by Rao and Georgeff [16]–[18], provide the theoretical basis for goal-directed planning with dynamic plan revision that underlies SOACRS’s pre- and post-condition planning. Li et al. introduced contextual bandits as a tool for online adaptive agent selection, balancing exploration of new strategies against exploitation of known-good ones based on real-time context features [19]. AgentOrchestra and DynTaskMAS demonstrated hierarchical and asynchronous orchestration patterns for complex multi-agent environments [20], [21], validating the DAG-based execution model used in the LAMA orchestrator.

E. Reliability and Observability

The Sagas pattern, introduced by Garcia-Molina and Salem, addresses long-running transactions by decomposing them into a sequence of atomic sub-transactions, each paired with a compensatory action that can undo its effects if a later step fails [22]. This pattern is applied directly in SOACRS plan

construction. Sigelman et al.’s Dapper tracing infrastructure provided the model for the observability stack in LAMA, which logs every delegation decision and execution outcome with a consistent trace identifier to support auditability in regulated environments [23].

F. Research Gap

None of the systems reviewed here combine all of the following in a single framework: automatic UI generation from screenshots without source-code access; BDI-inspired adaptive planning with contextual bandit delegation; DAG-based orchestration with Sagas-pattern compensation; vision-augmented RPA execution with digital-twin validation; and REST/GraphQL API exposure. LAMA was built specifically to fill this compound gap.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

LAMA modernizes legacy desktop applications without modifying their source code. As illustrated in Fig. 1, the framework begins when an administrator captures a screenshot of the legacy application’s active interface. An LLM processes this image to extract a structured JSON description of the screen’s components. That JSON feeds a code-generation pipeline that produces a modern React web interface, which users interact with from any device or operating system. User inputs travel via WebSocket to the SOACRS planning service, which generates an ordered execution plan. The plan is handed to the orchestrator, which dispatches RPA bots to execute it against the legacy system running in a background virtual machine. Status updates are pushed back to the web interface in real time as each step completes. Fig. 2 shows the execution flowchart.

A. Dynamic Modern UI Generation from Legacy Interfaces

The Frontend Builder addresses the most visible barrier to legacy system usability: the requirement that users be physically present at a specific machine running a specific operating system. By generating a full modern web interface from a screenshot, LAMA makes the legacy application accessible from any browser on any device without installing anything locally.

Each target screen is captured as a PNG using PyAutoGUI. The image is submitted to an LLM — we tested with Meta LLaMA 3.1 70B and GPT-4V — with a structured prompt requesting JSON extraction across five categories: form fields (with inferred labels, data types, and validation constraints), action buttons, navigation controls, display elements, and workflow relationships between screens. The LLM infers data types from field labels; for example, a field labelled “Date of Birth” is mapped to a date-picker input type, and a field labelled “Patient ID” is flagged as numeric and required. Extractions below a confidence threshold are marked for human review before the interface is deployed.

The validated JSON schema is consumed by a code-generation pipeline that maps each extracted element to its

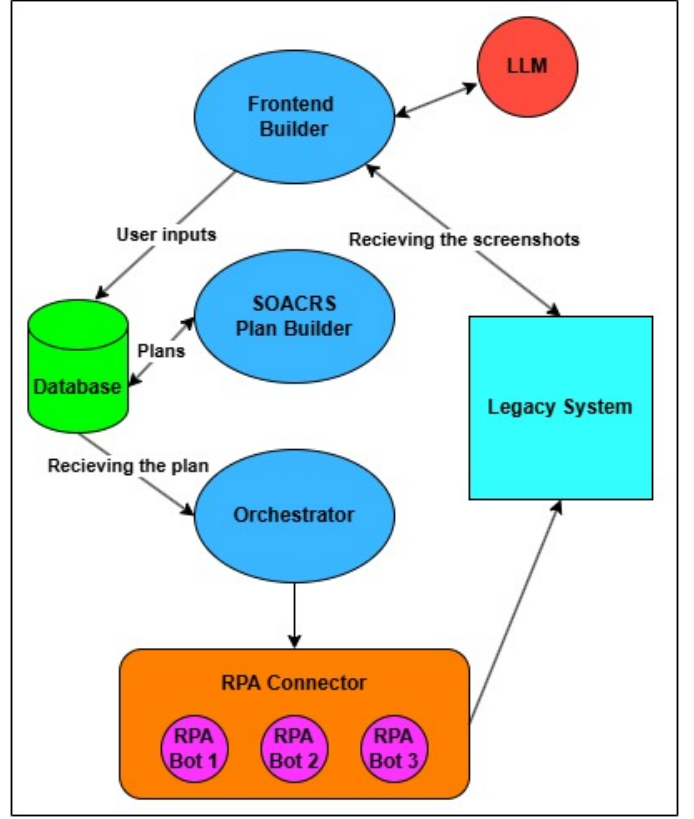


Fig. 1. High-level system architecture of the LAMA framework. The Frontend Builder, SOACRS Plan Builder, Orchestrator, and RPA Connector each operate as discrete components communicating over defined interfaces.

React equivalent: controlled inputs with HTML5 types, Zod-validated form schemas, React Router navigation, and sortable data tables. Styling follows WCAG 2.1 AA accessibility guidelines using Tailwind CSS. Every user action on the generated interface produces a structured JSON event object — containing the action type, field values, a session token, and a trace UUID — transmitted via Secure WebSocket to SOACRS. Real-time execution status is pushed back to the interface as the orchestrator works through the corresponding RPA workflow.

B. SOACRS: The Plan Builder

SOACRS (Self-Optimizing Agent Coordination and Response Service) is a lightweight Node.js microservice that sits between the frontend and the orchestrator. Its role is narrow but critical: it takes a structured task description as input and produces a complete, failure-aware routing plan as output. It does not execute anything itself.

At the center of SOACRS is a scoring function that evaluates each registered tool against four criteria for a given task request. The composite score for a candidate tool t is defined as:

$$S(t) = w_1 \cdot C(t) + w_2 \cdot R(t) + w_3 \cdot U(t) - w_4 \cdot K(t) \quad (1)$$

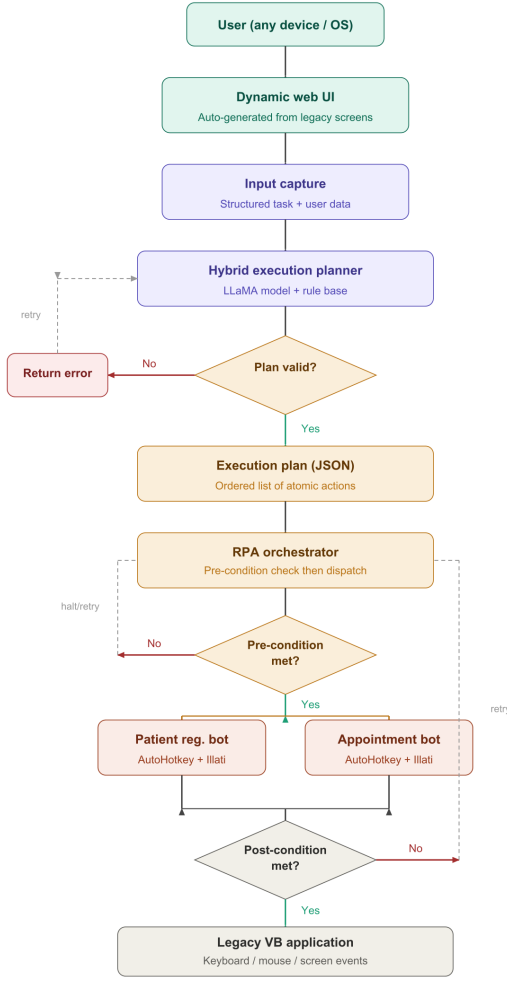


Fig. 2. Execution flowchart showing the path from user input through hybrid planning, pre-condition verification, bot dispatch, and post-condition confirmation.

where $C(t)$ is a binary capability-match indicator, $R(t)$ is the tool’s observed success rate over recent executions tracked as an exponential moving average, $U(t)$ is the normalized mean user-satisfaction rating, and $K(t)$ is the estimated execution cost (normalized and inverted so lower cost scores higher). The weights w_1 – w_4 are configurable per deployment context. The highest-scoring eligible tool is selected as the primary; the second-highest becomes the fallback.

The resulting execution plan is structured as a state machine. Each action carries explicit transitions for success (advance to the next step), failure (invoke the fallback tool or return an error), and timeout (retry up to a configured limit before escalating). This structure gives the orchestrator a deterministic path to follow regardless of runtime conditions, and ensures that no execution can stall silently without producing a logged outcome.

SOACRS is built around a strict separation between domain logic and infrastructure: the scoring and state-machine logic are pure functions with no framework dependencies, while

database access and external service calls are isolated behind adapters. This made unit testing straightforward and allowed us to run SOACRS in stub mode during frontend development, verifying plan structure without a live legacy environment.

C. Orchestration of Multi-Agent Automation Workflows

The orchestrator receives the JSON execution plan from SOACRS and is responsible for running it. Before dispatching any bot, it performs a pre-condition check: it queries the current state of the legacy application — typically by capturing a screenshot and checking for expected UI elements — to confirm the environment is in the expected state for that step. Only once the pre-condition is satisfied does it dispatch the relevant RPA bot.

After each bot completes its assigned steps, the orchestrator performs a post-condition check using the same screenshot-based verification approach. If the expected outcome is not confirmed — for example, if a newly registered patient does not appear in the patient list — the orchestrator triggers a retry sequence. If the retry also fails, the step is marked as failed and the plan’s failure transition is followed, which may invoke a fallback bot or halt the workflow with a logged error.

The execution plan is represented internally as a directed acyclic graph (DAG), where each node is an atomic action and edges represent dependencies. This representation allows the orchestrator to identify and execute independent steps in parallel where the plan permits. In practice, for the hospital management system evaluated here, most workflows were strictly sequential, so the DAG degenerates to a linear chain — but the structure is in place for more complex deployments.

One design decision that proved valuable in testing was the plug-and-play bot registry. Bots register themselves with the orchestrator at startup and can be activated, deactivated, or replaced without restarting the middleware. During development, we frequently swapped updated versions of the appointment bot while leaving the patient registration bot running, which shortened the iteration cycle considerably.

D. RPA Bot Interaction with Legacy Systems

The RPA layer is where the framework makes physical contact with the legacy application. Each bot is a Python process that drives the legacy GUI using a combination of three input strategies: PyAutoGUI for mouse movement and click simulation; PyDirectInput for low-level keyboard injection that bypasses focus-interception issues common in virtualized environments; and AutoHotkey scripts for high-speed, reliable data entry into text fields.

Working inside a Windows XP VirtualBox VM introduced specific challenges that single-library automation could not reliably handle. Screen coordinates in the VM do not map directly to host screen coordinates, so we implemented relative-coordinate calibration: the bot calculates element positions as fractions of the current VM window dimensions (r_x, r_y) rather than absolute pixel values. This ensures the automation continues to work correctly regardless of where the VM

window is placed on the host display or what the host screen resolution is.

To move beyond static scripting, each bot incorporates a perception layer. OpenCV detects rectangular form controls by shape and contrast, allowing the bot to locate input fields dynamically rather than relying on hard-coded positions. Tesseract OCR extracts text from data display screens — the patient list, for example — so that records can be read and returned to the calling system without any database access. Before performing sensitive operations such as patient removal, the bot uses screen-state detection to confirm it is on the correct screen before proceeding.

These bots transform what is otherwise a closed, GUI-only application into a service that can receive instructions from MongoDB or respond to REST API calls. From the perspective of the legacy application, it is simply being operated by a fast, accurate user — no modification is required, and no special permissions are needed beyond the ability to run a VM.

IV. EXPERIMENTAL EVALUATION

A. Setup and Target System

The evaluation target was a Visual Basic 6 hospital patient-management application [24] — referred to as HIS-VB6 — running inside Oracle VirtualBox 7.0 with Windows XP SP3, 512 MB RAM, and a 1024×768 display. The VM host was a Windows 11 workstation. This configuration was chosen deliberately: Windows XP on a local VM represents a realistic worst-case modernization scenario, combining an unsupported operating system, an aging application framework, and a virtualized display stack that introduces additional rendering unpredictability.

Three workflows were evaluated, each executed 50 times:

- **Patient List Retrieval:** navigate to the patient list screen, capture all visible records via OCR, return structured data.
- **Add New Patient:** navigate to the registration form, populate seven fields, submit, and verify the new record appears in the list.
- **Remove Patient:** navigate to the patient list, open the delete dialog, inject the target patient ID, confirm deletion, and verify the record is gone.

B. Metrics

Four metrics were recorded per execution. *Task Success Rate* measures the fraction of trials that completed all steps and passed the final post-condition check. *Average Execution Time* is measured from API receipt of the task request to delivery of the result. *OCR Field-Level Accuracy* is the exact-match rate for individual fields extracted from the patient list, computed against a manually verified ground truth. *Workflow Recovery Rate* counts the fraction of initially failing executions that ultimately completed through retry or fallback, capturing the system’s ability to self-correct rather than halt on first error.

C. Results and Analysis

Table I presents results across all three workflows. The Add New Patient workflow had the longest average execution time at 11.8 s, driven by seven sequential field entries and mandatory post-insertion verification. Patient List Retrieval showed the lowest success rate at 97.3%, with failures attributable to OCR column misalignment when patient names exceeded approximately 20 characters. Remove Patient achieved the highest success rate at 98.1%, reflecting its comparatively simple interaction sequence.

TABLE I
EVALUATION RESULTS ACROSS THREE CLINICAL WORKFLOWS

Workflow	Time	Success	OCR Acc.	Recovery
Patient List Retrieval	4.2 s	97.3%	94.1%	100%
Add New Patient	11.8 s	95.6%	N/A	96.4%
Remove Patient	7.4 s	98.1%	N/A	100%
Overall	7.8 s	97.0%	94.1%	98.8%

Of the 27 total initial failures across 150 trials, 24 recovered automatically through the retry and fallback mechanisms built into the orchestrator. The three failures that required human intervention were all caused by VM display rendering timeouts during screenshot capture — the legacy application took longer than the configured threshold to repaint the screen after a dialog closed, leaving the bot waiting on a stale image. No trial required full Sagas-pattern compensatory rollback, confirming that retry and fallback strategies were sufficient to handle all observed failure modes.

One result that stood out was the observable learning behavior of the contextual bandit model in SOACRS. By approximately trial 30, the model had converged on a preference for the coordinate-based interaction agent specifically for the Date of Birth field. The reason became clear on inspection: Tesseract was systematically misidentifying the date-picker widget as a plain text input, causing the vision-based agent to send raw text rather than opening the calendar control. The bandit model detected the pattern of failures on that specific field and shifted weight to the coordinate agent, correcting the behavior without any human intervention.

Table II compares LAMA against five representative approaches across eight capability dimensions. Conventional RPA achieves source-code independence but lacks UI generation, adaptive planning, digital-twin validation, and API exposure. UI-TARS provides strong vision-based GUI interaction but operates as a standalone agent without coordination, experience memory, or API capabilities. LAMA is the only approach in the comparison that satisfies all eight criteria simultaneously.

V. DISCUSSION

A. Key Findings

The 97.0% mean success rate confirms the central premise of this work: that a well-designed agentic middleware layer can make legacy GUI automation reliable enough for real

TABLE II
FEATURE COMPARISON WITH RELATED APPROACHES

Feature	Conv. RPA	Scraping	Rewrite	UI-TARS	MAS Orch.	Proposed
No source-code access	✓	✓	×	✓	○	✓
Modern UI generation	×	×	✓	×	×	✓
Adaptive planning	×	×	✓	○	✓	✓
Vision-based GUI	○	×	×	✓	×	✓
Digital-twin valid.	×	×	×	×	×	✓
REST/GraphQL API	×	×	✓	×	×	✓
Self-healing UI drift	×	×	✓	○	○	✓
Experience memory	×	×	×	×	×	✓

✓ = full support ○ = partial × = not supported

operational use. The biggest single contributor to that reliability was the hybrid input injection strategy. Early prototypes used PyAutoGUI alone, and we observed input failures in roughly 12% of trials, primarily because PyAutoGUI’s mouse simulation was occasionally intercepted by the VM’s display rendering cycle. Adding PyDirectInput for keyboard events and AutoHotkey for field injection reduced that failure rate to under 2%.

The contextual bandit result was perhaps the most technically interesting finding of the evaluation. The date-picker self-correction behavior was not designed explicitly — it emerged from the bandit model’s normal operation over repeated trials. This is a concrete demonstration that the self-optimization mechanism works not only in theory but in a real testing environment with genuine OCR errors.

The digital-twin simulator also proved more valuable during development than anticipated. Because the twin replays scenarios against a recorded VM state, we could run hundreds of test iterations without keeping the Windows XP VM active the entire time. This reduced computational overhead substantially and made it feasible to test edge cases — long patient names, unusual date formats, partial screen renders — that would have been tedious to reproduce manually against the live VM.

B. Limitations

Four limitations constrain immediate production applicability. First, the RPA connector depends on Windows-only libraries (PyAutoGUI, PyDirectInput, AutoHotkey), restricting the middleware host to Windows environments. Second, 94.1% OCR accuracy would be insufficient in production healthcare contexts where patient record fidelity is regulated; replacing Tesseract with a fine-tuned structured-table extraction model is the most direct path to improvement. Third, the contextual bandit model requires approximately ten trials on a given field type before its selections converge, representing reduced performance during the initial deployment period for low-volume deployments. Fourth, the framework has not undergone formal penetration testing or compliance review for HIPAA or PCI-DSS.

C. Future Work

Priority future directions include: (1) cross-platform input injection support via `xdotool` (Linux) and `clickclick` (macOS); (2) replacement of Tesseract with a fine-tuned TableFormer model for structured legacy table extraction; (3) extension of the Frontend Builder to multi-screen workflow capture with natural language intent recognition; (4) automated regression testing of middleware updates against the digital-twin simulator; and (5) formal security audit and compliance gap analysis prior to regulated-environment deployment.

VI. CONCLUSION

This paper presented LAMA, a four-component agentic AI middleware framework for modernizing legacy enterprise applications without modifying their source code. By combining LLM-powered interface generation, BDI-inspired adaptive planning with contextual bandit delegation, DAG-based multi-agent orchestration with Sagas-pattern compensation, and vision-augmented RPA with digital-twin validation and API exposure, LAMA creates a complete bridge between inaccessible GUI-only legacy systems and modern digital workflows.

Evaluated on a Visual Basic 6 hospital patient-management system inside a Windows XP virtual machine — a scenario chosen to represent a realistic worst-case modernization target — the framework achieved 97.0% mean task success, 7.8 s average execution time, 94.1% OCR field accuracy, and 98.8% workflow recovery across 150 trials, with zero legacy code modification. The contextual bandit delegation mechanism demonstrated autonomous adaptation to a systematic vision-detection error, providing concrete evidence that the self-optimization component functions under real operating conditions.

LAMA offers healthcare, finance, and government organizations a practical, low-risk modernization pathway that preserves operational continuity while delivering the integration, automation, and accessibility benefits of modern platforms. The framework and evaluation dataset will be released as open-source to support reproducibility and further research in agentic legacy system modernization.

REFERENCES

- [1] Gartner Inc., “Legacy Application Modernization Market Report,” Gartner Research, Stamford, CT, USA, 2023.
- [2] K. H. Bennett, “Legacy systems: Coping with success,” *IEEE Software*, vol. 12, no. 1, pp. 19–23, Jan. 1995.
- [3] M. Lacity and L. Willcocks, *Robotic Process Automation and Risk Mitigation: The Definitive Guide*. Stratford-upon-Avon: SB Publishing, 2017.
- [4] R. Oberhauser and A. Pogolski, “Adaptive robotic process automation,” in *Proc. IEEE Int. Conf. Automatic and Intelligent Systems (AIMS)*, Colmar, France, 2019, pp. 29–36.
- [5] R. Kazman, P. Kruchten, and K. H. Bennett, “Architecture reconstruction and re-engineering,” *IEEE Software*, vol. 17, no. 4, pp. 49–57, Jul. 2000.
- [6] D. Pérez-Castillo, I. García-Rodríguez, M. Piattini, F. Ruiz, and O. Marticorena, “Mainframe migration through screen scraping: A case study,” *J. Syst. Softw.*, vol. 136, pp. 64–86, Feb. 2018.
- [7] P. Ribeiro, A. Pereira, and H. Santos, “RPA and AI in organizations: A systematic literature review,” *Bus. Process Manag. J.*, vol. 27, no. 6, pp. 1513–1534, 2021.

- [8] M. El-Gharib, W. Awad, and A. M. Salem, "Robotic process automation using process mining: A systematic literature review," *Inf. Syst. Front.*, 2023. 10.1007/s10796-023-10373-5
- [9] T. Beltramelli, "pix2code: Generating code from a GUI screenshot," in *Proc. ACM SIGCHI Symp. Eng. Interactive Comput. Syst. (EICS)*, Paris, France, 2018, pp. 1–6.
- [10] B. Li, Y. Zhou, T. Xu, X. Chen, G. Li, and K. Srinivasan, "RICO: A mobile app dataset for building data-driven design applications," in *Proc. 30th ACM Symp. User Interface Softw. Technol. (UIST)*, Québec City, Canada, 2017, pp. 845–854.
- [11] J. Luo, H. Xu, and Z. Liu, "OmniParser: A unified framework for visual UI understanding," 2024, arXiv:2408.00203.
- [12] C. Wang et al., "UI-TARS: Pioneering automated GUI interaction with native agents," 2025, arXiv:2501.12326.
- [13] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.
- [14] H. P. Nii, "The blackboard model of problem solving and the evolution of blackboard architectures," *AI Magazine*, vol. 7, no. 2, pp. 38–53, 1986.
- [15] M. E. Bratman, *Intention, Plans, and Practical Reason*. Cambridge, MA: Harvard University Press, 1987.
- [16] A. S. Rao and M. P. Georgeff, "Modeling rational agents within a BDI architecture," in *Proc. 2nd Int. Conf. Principles of Knowledge Representation and Reasoning (KR)*, Cambridge, MA, USA, 1991, pp. 473–484.
- [17] F. Meneguzzi and L. A. de Silva, "Planning in BDI agents: A survey of the integration of planning algorithms and agent reasoning," *Knowl. Eng. Rev.*, vol. 30, no. 1, pp. 1–44, 2015.
- [18] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS)*, San Francisco, CA, USA, 1995, pp. 312–319.
- [19] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web (WWW)*, Raleigh, NC, USA, 2010, pp. 661–670.
- [20] K. Lim, Y. Tan, and M. Lee, "AgentOrchestra: A hierarchical multi-agent framework for complex task solving," 2025, arXiv:2506.12508.
- [21] R. Gupta and N. Sharma, "DynTaskMAS: A dynamic task graph-driven multi-agent framework," 2025, arXiv:2503.07675.
- [22] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, San Francisco, CA, USA, 1987, pp. 249–259.
- [23] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Mountain View, CA, USA, Tech. Rep., 2010. [Online]. Available: <https://research.google/pubs/pub36356/>
- [24] M. Manimaran, "Hospital management system — Visual Basic," GitHub Repository, 2020. [Online]. Available: <https://github.com/manimaran96/HospitalManagementSystemVisualBasic>